FACULDADE DE ENGENHARIA DA UNIVERSIDADE DO PORTO

LayerIt!
-Towards Developing Time Aligned Music Data Visualizations-

<Fernando Azeredo>

WORKING VERSION

Mestrado em Engenharia Informática e Computação

Supervisor: Prof. <António Sá Pinto>

June 30, 2026

Abstract

Music Information Retrieval (MIR) visualization tools play a crucial role in music analysis, annotation, and evaluation, but existing solutions often make it difficult to combine audio, symbolic scores, and algorithmic outputs in a single, time-aligned view. This thesis presents LayerIt, a Python-based object-oriented (Object-Oriented (OO)) framework for generating custom score-aligned MIR visualizations by combining established libraries and tools, including librosa, matplotlib, Modusa, ScoreWarp, and BeatThis. LayerIt is built around a modular layer-composition model in which data sources, audio plots, score renderings, and algorithmic outputs are treated as independent components that can be assembled into tailored visualizations. The resulting figures are exported as Scalable Vector Graphics (SVG), enabling resolution-independent output, post-processing, and future integration with web-based or interactive applications. As a validation case, LayerIt is used to visualize beat tracking results alongside aligned score and audio representations, showing how the framework supports both beat annotation and beat tracking evaluation through discrete beat estimates and continuous confidence curves. More broadly, the project demonstrates a reusable foundation for future specialized MIR visualization workflows.

Keywords: Music Information Retrieval • Visualization • Audio-to-Score Alignment • Beat Tracking • SVG • Object-Oriented • Python

Acknowledgements

<Fernando Azeredo>

Contents

List of Figures

List of Tables

List of Acronyms

BA Beat Annotation
BT Beat Tracking
DAW Digital Audio Workstation
GUI Graphical User Interface
HTML HyperText Markup Language
JS JavaScript
MEI Music Encoding Initiative
MIDI Musical Instrument Digital Interface
MIR Music Information Retrieval
ML Machine Learning
OO Object-Oriented
PP Piano Precision
PNG Portable Network Graphics
SOTA State of the Art
SV Sonic Visualiser
SVG Scalable Vector Graphics
XML Extensible Markup Language

Chapter 1

Introduction

1.1 Contextualization

Music Information Retrieval (MIR) is a research field that develops computational methods to analyze, interpret, and retrieve musical information from performance recordings and symbolic representations. MIR bridges a wide range of disciplines, including musicology, signal processing, machine learning, and human-computer interaction. As a data-intensive field, visualization plays a crucial role in MIR research, the how and what is visualized, shapes the conclusions we draw and guides further progress.

The challenge of visualization in MIR is particularly significant because music is an inherently abstract and subjective medium. Translating music into the visual domain is a problem that spans millennia; throughout history, multiple systems of musical notation have been developed in attempts to capture sound in written form, yet this remains an evolving challenge. In a MIR context, this means that musical data comes in a variety of forms:from audio recordings, to symbolic scores, to annotations of expressive qualities. Each of these require different visual approaches. An audio visualization such as a waveform or spectrogram reveals how a performance sounds, while a musical score conveys how a piece should be performed; both serve distinct and important purposes for different research tasks.

MIR research increasingly demands visualizations that integrate multiple modalities simultaneously. MIR researchers often need to inspect algorithmic outputs alongside audio representations and symbolic scores in a time-coherent manner. This kind of multi-modal, time-aligned visualization is central to both the evaluation of MIR algorithms and the development of annotation workflows. Audio-to-Score Alignment (i.e., the task of synchronizing a performance recording with its corresponding symbolic score) is a key enabler of this type of visualization, as it provides the temporal anchor needed to align heterogeneous data sources into a unified view.

1.2 Developed Project

For this thesis we developed LayerIt, a Python-based Object-Oriented (OO) tool designed to generate custom time-aligned visualizations of audio, symbolic music representations, and algorithmic data. LayerIt integrates a set of well-established MIR libraries and tools namely: librosa, matplotlib, modusa, BeatThis, and ScoreWarp. into a unified, modular framework. The result is an Scalable Vector Graphics (SVG) based visualization pipeline where visual elements are treated as independent layers that can be composed, reordered, and combined to produce tailored visualizations suited to specific research needs.

LayerIt's core contribution is the integration of score-aligned audio visualizations with overlaid algorithmic outputs. By leveraging ScoreWarp (a tool that physically shifts score elements along a time axis to match the temporal positions of a musical performance) LayerIt is able to align the symbolic score with spectrograms, Beat Tracking (BT)and onset data within the same visualization. This is achieved through an SVG-based system in which each layer is a self-contained component that can be freely composed with others. This follows a consistent "layers within layers" design philosophy which works at different levels of interaction enabling different forms of interacting with visualizations generated or created through LayerIt.

The development of LayerIt followed an iterative process, passing through five distinct project iterations before arriving at the final solution. These ranged from web-based beat annotation interfaces to forks of existing Graphical User Interface (GUI) tools, each providing key insights that shaped the final direction of the thesis. This journey led to a deliberate focus on the visualization problem specifically, recognizing that a reusable, code-based Python tool built on commonly used MIR libraries would offer the greatest value to the community. LayerIt is designed with extensibility and integration in mind, so that it may serve as a foundation for future MIR visualization frameworks or be adopted into existing research workflows.

1.3 Motivation

This thesis was initiated by two closely related challenges within BT and Beat Annotation (BA) research. Contemporary BT systems are predominantly deep learning-based thus, the field is deeply reliant on tools to enable better execution of annotation and evaluation tasks, these are essential for advancing BT research.

Annotation tasks, and more concretely BA itself, remain a laborious and time-consuming process. Existing GUI tools such as Sonic Visualiser (SV) present a clear gap in this area: integrating newer state-of-the-art BT algorithms is difficult, and BA-specific features have seen little meaningful innovation. This disconnect between annotation workflows and the algorithms they are meant to support is a concrete obstacle to progress in both domains.

For evaluation tasks, GUI tools suffer from a similar problem, where methods to visualize how developed algorithms perform on a deeper level are lacking. Providing better tools to evaluate and compare models between each other is also key to improving them.

Investigating this problem led us to identify a broader gap between modern MIR GUI tools and contemporary code-based research workflows. Although solutions like SV are powerful and widely used, their development cannot keep pace with the rapidly evolving MIR field, integrating new algorithms or producing specialized visualizations is often impractical within them. A particularly under served case is the visualization of score-aligned data: combining audio and data representations with a musical score typically involves manual image manipulation, with no unified methodology in the community. Modern BT and BA research increasingly demands visualizations that go beyond any single existing tool, such as aligning algorithmic probability outputs with both a performance's audio and its symbolic score. LayerIt was developed to address this need in the BT and BA tasks. Through our approach, we intend to demonstrate how LayerIt may be useful for other MIR research fields.

More broadly, this thesis is motivated by the belief that visualization infrastructure is a relevant concern in MIR research. The ability to see and inspect data (i.e., from raw audio to algorithmic inner workings) is inseparable from the ability to understand, evaluate, and improve MIR systems. Researchers frequently resort to *ad hoc* solutions that lack reproducibility and are difficult to share or extend, a fragmentation that represents a real obstacle to the collaboration which MIR research depends on. By contributing a flexible, extensible, and community-oriented visualization tool built on established libraries, this thesis aims to not only address an immediate practical gap, but also to establish a foundation for a framework that can support the growing complexity and interdisciplinarity of MIR research.

1.4 Thesis Structure

The remainder of this thesis is organized as follows:

- **Chapter 2** provides a literature review covering the main audio visualization types, along with existing GUI and code-based visualization tools, and the task of Audio-to-Score Alignment.
- **Chapter 3** presents the problem in detail and traces the development of LayerIt through five project iterations, culminating in the development of LayerIt. In this chapter we'll also present the rationale behind key concepts and features present in LayerIt.
- **Chapter 4** describes the implementation of LayerIt, covering the layer alignment method, the "layers within layers" philosophy, and its application to BT and BA visualization tasks.
- **Chapter 5** reflects on the achievements of the thesis and outlines directions for future work.

Chapter 2

Literature Review

MIR is the field of research that focuses on the development of algorithms and systems to analyze, retrieve, and organize music data. MIR bridges a wide range of disciplines, including: musicology, signal processing, machine learning, and human-computer interaction [10]. In this chapter we'll provide a comprehensive review of Music Information Retrieval (MIR) visualization techniques and tools, establishing the foundation for understanding the problem we address in this thesis.

We begin by exploring the landscape of music visualization methods, categorizing them into three main types: sheet music representations, symbolic machine-readable formats, and audio-based visualizations. This foundation is essential for understanding how different types of musical information can be represented visually and interacted with computationally. We then examine contemporary MIR visualization tools, distinguishing between GUI-based solutions and code-based workflows to highlight the gap that motivates our work. Finally, we introduce Audio-to-Score Alignment, the core task central to this thesis, and explore relevant tools and approaches that have informed our solution.

2.1 Visualization Types

We can sum up all music related visualizations into three categories:

1. Sheet Music and Score Images
2. Symbolic Representations
3. Audio Representations

2.1.1 Sheet Music and Score Images

Sheet music describes musical work using a formal language based on musical symbols and letters, thus, this kind of music representations always require a level of music literacy from the performing musician. Furthermore, sheet music usually does not induce explicit information on how a given piece should be performed, it is intended for the musician to have a certain level of

interpretation liberty. This can include the variance of tempo, articulation and dynamics among others.

Sheet music also comes in a variety of forms that vary throughout History and cultures. For our purposes we are going to focus on the western standard of music notation. Still, within western music, there can be vastly different forms of notation, however, most are based on a staff which sets five horizontal lines and four spaces, each representing a different musical pitch. Musical relevant symbols are put inside the staff to denote pitch, temporal and expressive indications. [10]

2.1.2 Symbolic Representations

Symbolic Representations are machine-readable formats where musical events and expressions are explicitly encoded. These come in various formats, such as Musical Instrument Digital Interface (MIDI) and MusicXML.

Visually we can identify two main applications for this category, these include: piano-roll representations and digital score representations.

Historically piano-roll representations come from so-called “player pianos”, these were self playing pianos that became quite popular in the late 19th to early 20th century. This was achieved by a mechanism that would hit keys of the piano through a continuous roll of paper with perforations. The roll would move over a reading system (known as a tracker bar) that would trigger the musical actions accordingly.

This concept of a continuos and explicit representation of music performance would later prove useful for digital applications. Such is the case for the Piano-Roll.

Today the **Piano-Roll** is a staple feature on any Digital Audio Workstation (DAW) as a visual and interactive tool to write and edit musical events in the form of MIDI notes and attributes (such as velocity and other expressive controls).

MusicXML refers to the Extensible Markup Language (XML) based language to transcribe score music into the digital domain. There are multiple sub-variants of the MusicXML format for specific applications and there are other formats for digital scores, but MusicXML can be considered as a "universal" format accepted in most digital score editors.

MusicXML was developed to serve as a universal format for storing and sharing music scores across different music notation applications. Music notation digital editors then employ proprietary variants of this format for addressing the program’s specific characteristics. In some cases these may not even be xml based, but MusicXML format can be considered the universal format. These include, among others:

- **.mscz and .mscx**: These are Musescore specific formats named for "MuseScore Format" and "Uncompressed MuseScore" format respectively. Both xml based.
- **.dora** - "Dorico" specific format (non xml based)
- **.sib** - "Sibelius" proprietary format of company Avid (also non xml based)
- **.mei** - The Music Encoding Initiative (MEI). XML based.

The .mei format is the one that is essential for this thesis and the most useful for MIR purposes. MEI stands for both the XML-based score format in itself (.mei), and the community that develops it. Citing from the MEI website's introduction:

"The Music Encoding Initiative (MEI) is a community-driven, open-source effort to define a system for encoding musical documents in a machine-readable structure. MEI brings together specialists from various music research communities, (...) to define best practices for representing a broad range of musical documents and structures."[1]

Important to note that MEI focuses on a "machine-readable structure". This structure relies on unique identifiers (@xml:id) that make every element in the score individually addressable, from individual noteheads to complex spanning elements like slurs and ties. However, more than a music score format, MEI also serves to keep various notation, performance and interpretation data not only stored but also linked to individual score elements. This is why the MEI format and it's community are so essential to MIR as a whole.

That being said, MEI is a purely data driven project, and thus in order for it to be visualized or interacted with it needs to be decoded into a "human-readable format". Thats where Verovio comes in. The MEI and Verovio projects are closely inter-linked for that reason. While MEI defines how musical information is stored, Verovio is the primary tool used to transform that data into interactive visualizations and performance-ready outputs.

Verovio is a comprehensive open-source library designed to serve as a native typesetting engine for the MEI format. Verovio's primary role is to engrave MEI data directly into SVG without losing rich metadata through intermediate format conversions. The choice for making it into an SVG is an essential one for multiple reasons, quoting from Verovio's reference book:

> "In a web environment, the addressability can be used for highlighting graphical elements such as notes or any other music symbols. One additional characteristic of SVG is that its XML tree can be structured as desired, and an innovative design feature of Verovio was to go further in the structuring of the output by leveraging this characteristic. Since Verovio implements the MEI structure internally, this key feature of SVG made it possible to preserve the MEI structure in the design of the SVG output in Verovio. Preserving the MEI structure in the SVG output is a considerable overhead in the rendering process but makes it a unique feature of Verovio.
>
> As a result, Verovio output in SVG is not the end of a unidirectional rendering process. Quite on the contrary, it should instead be seen as an intermediate layer standing between the MEI encoding and its rendering that can act as the cornerstone for a bi-directional interaction: from the encoding to the notation, but also from the notation to the encoding through the user interface." [13]

To further demonstrate the usefulness of the MEI and Verovio, we'll present next a quote from the article relative to Piano Precision (PP) a GUI software for audio-to-score visualization, we will explore this tool further, for now the importance of this quote comes from demonstrating a practical application of MEI and Verovio.

> "Technically, what makes MEI especially useful is that it can be rendered to graphical SVG output while retaining its semantically meaningful XML hierarchy and unique identifiers for every score element (e.g., ties, notes). Piano Precision renders the scores into SVG output using Verovio, an open-source toolkit for visualizing MEI data. This makes it easy to locate interesting objects in a score and, e.g., highlight them in the rendering. The result is interactive images in which users can flip pages, zoom in and out of a score, and click on elements in a score to jump to their playback positions. " [5]

2.1.3 Audio Representations

Audio representations are graphs and/or plots that intend to represent sound. These may not be for strictly representing music, the intent is to translate sound from the audible to the visual domain. Audio is a term that refers to an acoustic sound that is turned into an electrical signal. Most audio representations, are thus digitally generated through applying mathematical equations to the audio signal, such as the Fourier Transform.

We can infer at least 3 components that define a given audio. These are, time, frequency and amplitude/energy. An audio visualization will present explicit information of at least 2 of these.

Waveforms are the most common and one of the simplest ways to represent audio. It consists of a single line on a graph in a 2D plot where amplitude is the y-axis and time is in the x-axis. This representation does not provide frequency information, it only shows how amplitude of the audio changes through time.

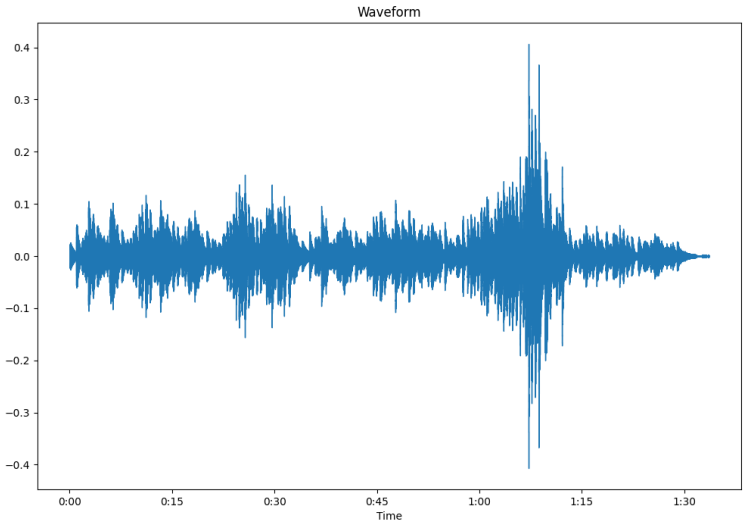

Figure 2.1: Example of an audio waveform.

A spectral visualization, or a spectrum, refers to a graph with only frequency and amplitude/energy representation in a given time window.

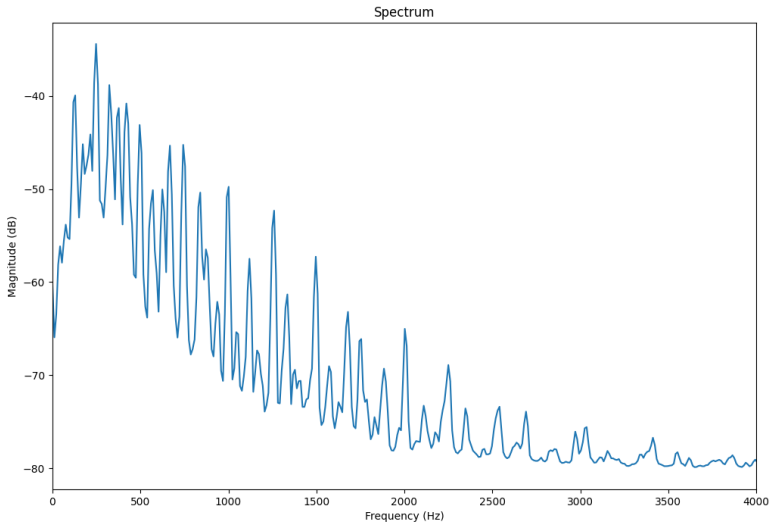

Figure 2.2: Example of audio Spectrum.

For displaying the three sound components the plot needs to represent three distinct dimensions. The spectrogram achieves this by having on the y axis the frequency, time on the x-axis and color as substitute for the third axis (amplitude), and in this way displaying all three parameters in a single 2D visual plot. There are multiple weights and variances of the spectrogram that have different applications.

The way frequencies are displayed in a given spectrogram can help understanding audio better for certain situations. There can be a wide variety of spectrogram representations, that can vary in how it's contents are displayed.

The linear spectrogram is the most basic form of spectrogram, where frequencies are displayed in a linear scale. Meaning, frequencies are equally spaced on the y-axis. For music applications, this kind of representation tends to be less useful, as it does not reflect the way humans perceive sound.

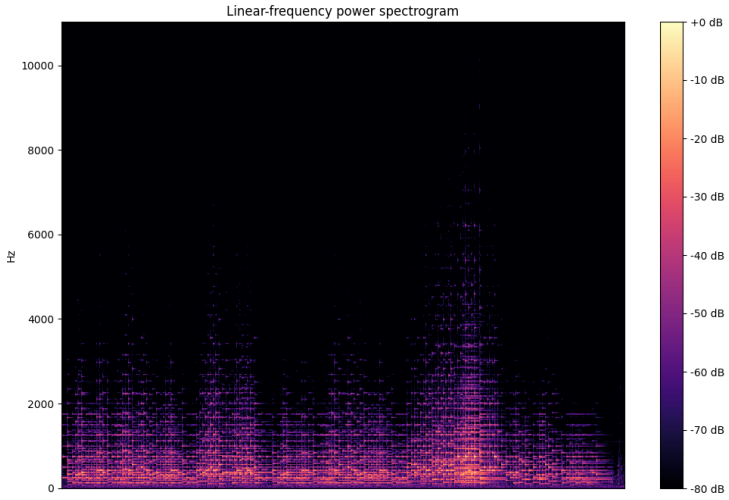

Figure 2.3: Example of Linear spectrogram.

The logarithmic spectrogram displays frequencies in a logarithmic scale. This mimics better the way humans perceive sound.

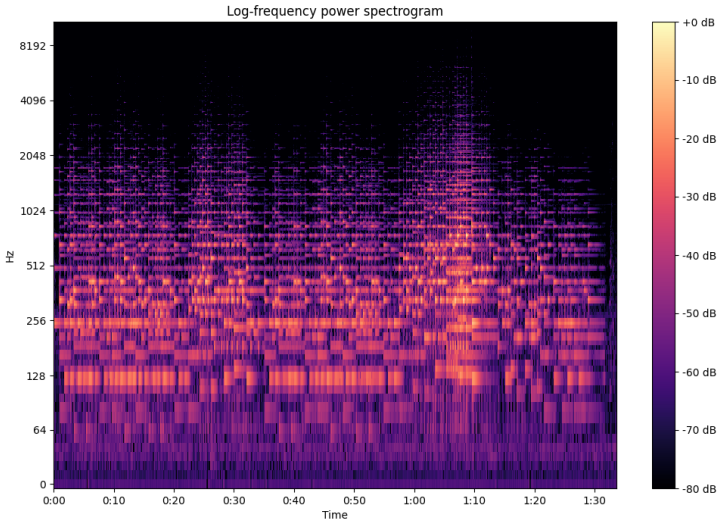

Figure 2.4: Example of Logarithmic spectrogram.

The Mel spectrogram also displays frequencies taking into account the human hearing curve. The difference being, that the logarithmic spectrogram simply organizes frequencies logarithmically. Whilst the Mel spectrogram organizes frequencies according to the Mel scale, which is a perceptual scale of pitches judged by listeners to be equal in distance from one another. For music related applications this type of spectrogram is much more useful, since we can more clearly distinguish in between musical pitches.

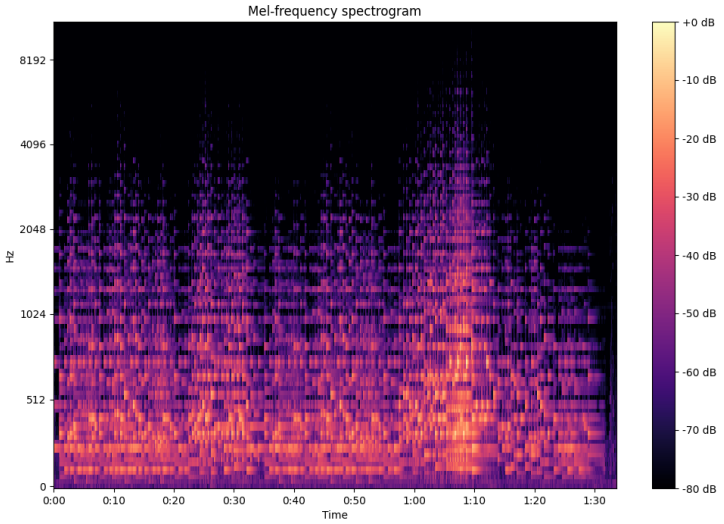

Figure 2.5: Example of Mel spectrogram.

Next I'll introduce the Constant-Q Transform (CQT) and the Chromagram. These are, to some extent, visual representations that derive from the spectrogram, the difference being that whilst the later shows frequency data, the CQT and chromagram can be used to visualize note data. To be exact, tone height in the case of the CQT and Chroma for the Chromagram. The following quote explains these terms accurately:

> "(...) a pitch can be separated into two components, which are referred to as tone height and chroma. The tone height refers to the octave number and the chroma to the respective pitch spelling attribute contained in the set {C, C♯, D,(...), B}." [10]

The CQT organizes its frequency content using geometrically spaced frequency bins. This is made in order to resemble the equal-tempered system. More commonly, the CQT can be interpreted as a Piano-Roll like representation.[15, p. 3]

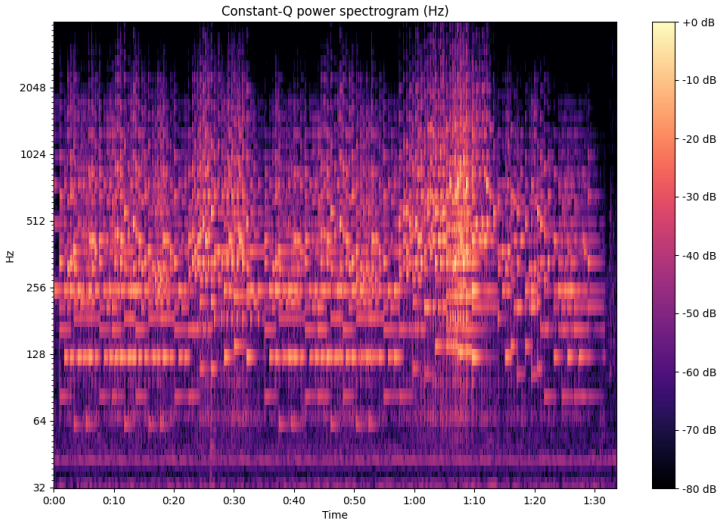

Figure 2.6: Example of a CQT.

While the CQT can distill the frequency spectrum into bins that correlate to individual notes, the chromagram takes this a step further by visually summing the energy of each note across all octaves. Thus in the end we get a plot divided into 11 lines, each representing a give note. In it's essence this type of visualization serves to see the usage of the same notes across the recording. A chromagram can be useful for determining harmonic and scale related data.

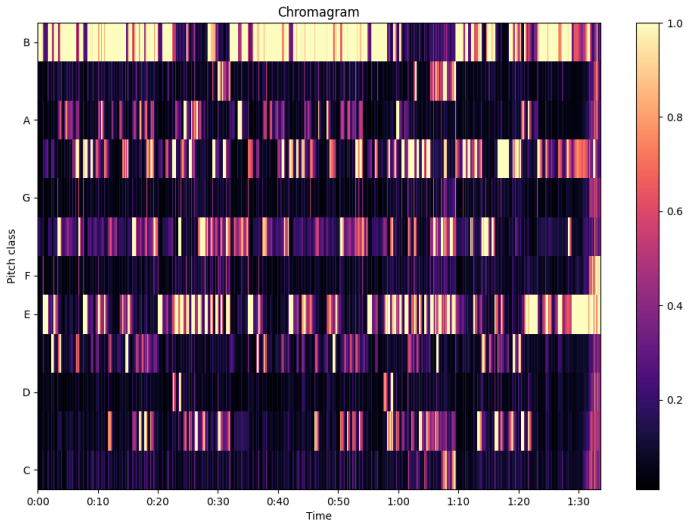

Figure 2.7: Example a Chromagram.

[6, 7, 10, 15]

2.2 MIR Visualization Tools

Within MIR we can identify two main ways for visualization of audio for MIR tasks, these are either by using GUI based tools, or by using programming-based workflows.

For GUI based tools two stand-out, these being Audacity and especially Sonic Visualizer (SV). Audacity is one of the most popular audio editors; it is designed for general audio related tasks from audio editing, recording, processing, and visualization. On the other hand, SV is a strictly designed program for audio visualization and analysis. Whilst Audacity the most "user friendly" and popular program, SV is meant for scientific and specialized audio visualization tasks.

Important to note that both software solutions are free and open-source based. These are important features, since they allow for users to easily access them, but also for researchers and experts to modify them and thus, contribute to the development of these and/or new tools.

One such example is Piano Precision (PP), a SV based GUI tool for visualizing score-aligned spectrograms.

> "The Piano Precision UI is adapted from Sonic Visualizer, with an additional area displaying a score (...) The interface is designed to help streamline the processes of loading a digital score and a performance recording, editing and visualizing annotations (...), and exporting score-aware annotations. "[5]

While modern GUI-based visualization tools may be powerful, they might not provide features that are required for state-of-the-art research and development. MIR research often requires highly specialized visualizations tailored to specific research questions that pre-built tools simply do not support. Understandably, the development of these programs simply cannot keep up with the constantly evolving advancements in the field.

That is why it is also quite common for researchers to develop *ad-hoc* programming-based methods for creating specialized visualizations. Researchers might need to integrate multiple data types (audio, scores, annotations, computational outputs) in ways that GUI tools cannot replicate.

Moreover, code-based workflows can be versioned, shared, and included in publications, ensuring reproducibility this is an essential requirement for academic research.

Another argument is that MIR is increasingly geared towards machine learning solutions and algorithms. The need to work with large music datasets and integrate machine learning applications for training, feature extraction, and other related tasks, explains the reliance on code-based visualization workflows.

Furthermore, the fact that MIR is fundamentally a computational research field more broadly justifies this approach.

Currently, Python is the go-to language for MIR research. Key libraries such as Librosa, Matplotlib, and Madmom, along with APIs such as Modusa and tools like Jupyter Notebooks, are common place in MIR.

2.3 Audio-to-Score Alignment

The task of Audio-to-Score Alignment (score matching or score alignment) consists of "synchronizing an audio recording of a musical piece with the corresponding symbolic score" [2]. There are multiple applications for such a feature, from automatic score following for music performance or for guided listening, to interactive navigation between audio and score (useful for audio editors) to MIR research and development.

There are two modes of alignment: **offline alignment** and **online alignment**. Offline alignment, consists of accessing a musical recording and performing the alignment afterwards, this can be useful for tasks such as musical analysis and audio editing. Online alignment (also known as also known as score following), processes the data in real-time while the audio signal is being received, this is useful for tasks like autonomous accompaniment of the score while a performance is occurring. Offline alignment is able to use the full length of the recording in order to determine the optimal alignment, this is because it can "look into the future" in order to establish the optimal matching. Because of this, Online alignment can't provide the same level of detail unless some deliberate latency is introduced into the system thus, online modes need to take into account precision trade-offs.

Besides online and offline modes of score matching, the alignment in itself can be done to various levels of the performance or score these include:

- Note-level
- Beat-level
- Segment-level

Note-level alignments are score matching processes that aim to synchronize at the individual note level. At it's core, note-level alignments are systems based on note and onset/offset detection. In MIR literature it can be also referred as "fine-grained alignment" due to the level of precision that these methods require. Contemporary Audio-to-Score MIR research focuses on this type of alignment not only because of the precision element but also because it can retrieve data that is relevant for other MIR fields and tasks. However, pure note-level alignments might falter if the performance deviates from the score (i.e. if the performer skips, repeats or adds notes or sections of the score). That's why some score matching solutions try to employ, on top of fine-grained alignments, other methods.

Piano Precision (PP) provides a note-level score aligned visualization. While the recording is playing PP shows the current playback position via highlighting the elements being played in the score in real time, note by note. This enables annotations to be mapped to both the audio and individual score elements simultaneously. The authors note the usefulness of this tool for performance researchers [5] however, we will make the case that such feature is useful for BT and MIR research more broadly

Beat-Level alignments aim to follow along beat and/or measures dictated in the score, and then linking them to the performance. Early score aligners [3] would rely on such methods. Today,

score alignment solutions, although mostly are note-level types, still combine Beat-Level alignment layers within their algorithms. A good example of this comes from [12], in it the authors improved on other alignment models, one strategy employed was adding analysis of "alignments at both the beat and the bar level". The beat level alignment would serve as "anchor points" in order to prevent "cascading errors, as misalignment will not propagate beyond the next anchor point", they noted that this improved results of automatic alignments.

Segment type alignments, map broad portions of performance audio to corresponding visual blocks in the score. This type of score matching is mostly made manually, it is common to see it in follow-along videos to help viewers listen to the music while visualizing the score. This type of alignment is considered "weak but musically meaningful" [6] since segments can be deliberately chosen i.e., to better show melodic or harmonic structure.

ScoreWarp is a relevant visual score alignment tool for this thesis. It presents a score that is tied to a physical time axis where, each element of the score is "physically shifted so that their location linearly coincides with the physical time of their sounding" [4]. ScoreWarp takes advantage of the MEI format and Verovio in order to automatically shift the position of musical elements, while maintaining the semantic integrity of the original score. In the next chapter we are going to explore this tool more in depth, since it became one of the key tools for the development of the project for this thesis.

2.4 Summary

This chapter provided a comprehensive overview of MIR visualization techniques and tools, establishing the foundation for understanding audio-to-score alignment.

We began by categorizing music visualizations into three main types: traditional sheet music representations, symbolic machine-readable formats, and audio-based representations. With this we established the importance of the MEI and Verovio as well as the usage of the SVG visual format in this context.

We then examined the contemporary landscape of MIR visualization tools, distinguishing between GUI-based solutions such as SV and PP, which provide user interfaces for audio analysis, and code-based workflows that afford researchers greater flexibility to develop specialized visualizations tailored to specific research questions. Python-based approaches using libraries like Librosa and Matplotlib have become the de facto standard for MIR research, particularly due to their integration with machine learning pipelines and reproducibility requirements.

Finally, we introduced one of the core tasks of this thesis: Audio-to-Score Alignment. This process synchronizes musical performances with their corresponding symbolic scores. ScoreWarp emerged as a particularly relevant tool, demonstrating how MEI and Verovio can be leveraged to create visually warped scores that maintain semantic integrity while aligning with performance time.

Chapter 3

Developed Solution

This chapter is dedicated to presenting and contextualizing the problem we chose to tackle and the subsequent solution we developed to address it.

The chapter begins by laying out the core problem and a brief presentation of the solution developed. After that, we'll present chronologically, the different project iterations we passed through, covering their motivations, achievements, why they were dropped and conclusions taken. Understanding where we came from provides essential context not only for this thesis, but also for understanding the core purpose and value statement of LayerIt.

Finally we'll close this chapter by presenting LayerIt: a Python-based OO tool for generating custom time-aligned visualizations of audio, music and algorithmic data. We'll also lay out core choices and design principles that define LayerIt and justify its value to the broader MIR community.

3.1 The Problem and our Solution

We identified a crucial gap between modern MIR GUI tools (i.e., SV) and programming-based workflows. SV, although being a widespread and a powerful tool, it falters regarding features for including many of contemporary MIR developments, besides this, SV's comprehensive and complex C++ codebase is difficult to manage and develop into. Because of this, MIR researchers depend on personal developed solutions to visualize their work. These can range from pure code-based solutions to other various creative methods:

- For code-based solutions, these might use a combination of different libraries and frameworks such as matplotlib, librosa, madmom and modusa to name a few. These can achieve interesting results but do require a certain level of programming knowledge and expertise to be developed effectively.
- Other researchers may rely on slicing, cropping, combining and drawing visual elements manually. From SVGs, Portable Network Graphics (PNG)s to digitalized PDFs there is no one unified and established framework in the MIR community for producing meaningful time-aligned visualizations.

An example of visualizations that usually are made using the latter method, are those that require an accompanying symbolic music representation (i.e., musical score). These ar mostly produced via manually cropping and manipulating scores in image formats, this can be a time consuming task. LayerIt intends to facilitate this process by providing tools that allow for alining scores with the required data and audio visualization layers.

Our proposed solution intends to take into account different methods of producing visualizations, from the pure code-based methods to editing SVG's manually. LayerIt was developed with the intent for it to be adaptable enough to be included into any given workflow or integrated in future frameworks or GUI tools.

3.2 Developed Iterations

This thesis started with the proposed title "Interactive Musical Beat Annotation GUI". The initial thesis proposition was to evaluate modern GUI software solutions used for BA (such as SV), and build upon them with a focus in BA practices. The proposed goals included:

1. Either develop a web-based annotation system, or advancing the existing SV framework. Emphasizing collaborative features, real-time feedback and build on interaction mechanisms for beat annotation.

2. Integration of Machine Learning (ML) self-supervised and human-in-the-loop features that provide intelligent annotation suggestions and improve user experience (e.g., Yamamoto et al., 2021 [16]).

3. Validation through user studies.

4. Publish the developed work as an Open Source Python library/framework. The purpose being, to contribute to MIR research community for further development and integration with other existing frameworks.

The motivation from this proposal came from mostly two identified flaws in this topic: The first is that, although the performance of beat tracking algorithms has been improving significantly in recent years, they mostly rely on deep learning based approaches. The issue with this, is that in order for these systems to be trained they need increasingly large and more complex datasets that, although not always directly [12], need human made and/or verified beat annotations. The second one is related to human made BA methods, that for the most part, still is an extensive and time consuming task. Besides that, although SV is a powerful tool, it presents a clear gap for beat annotation, where the integration of newer State of the Art (SOTA) BT algorithms is difficult and there have been little to no new added features that improve BA task significantly [16].

Summarizing, the thesis came from a focus in BT, BA and interactive GUIs to, in the end, focusing on a purely MIR visualization problem. Next we will present different iterations the development phase undertook. This should contextualize the change in scope and motivation of the thesis.

3.2.1 BeatMapJS

Developed from October 2025 until February 2026, BeatMapJS was our first attempt to create a GUI application for beat annotation. Our goal was to experiment and explore interactive BA features, libraries and frameworks while focusing on a web-based solution. Using JavaScript (JS)/HyperText Markup Language (HTML), BeatThis, and Wavesurfer.JS, we successfully built an interactive spectrogram with a web-based interface that integrated the BeatThis algorithm. The application included basic tools for beat annotation, region annotation capabilities, a table for editing tempo annotations and descriptions, and the ability to import and export beat annotations to .txt files (compatible with SV).

Although the program was functional, it was ultimately abandoned. The main reason was that it provided no significant innovative BA features beyond BeatThis integration, and critically, no Python-based library was produced as the backbone of the project, which was an important goal for ensuring the work could be reused by the community.

3.2.2 MEI-Basic-Edit

Running from February until March 2026, this project aimed to create a GUI ecosystem where users could manually adjust score elements while simultaneously viewing an audio visualization layer. We had identified an innovative BA feature, Audio-To-Score Alignment, and hypothesized it would greatly benefit both beat annotation and evaluation tasks. The project served as a learning opportunity to explore MEI and Verovio frameworks along with ScoreWarp.

Implemented in JS/HTML using Verovio, MEI score format, and SVG, the application allowed users to upload a recording and its corresponding MEI score file. A static spectrogram would be generated with the score displayed horizontally below it. Users could interact with individual score elements by clicking and dragging them horizontally to manually align the score with the audio-visual representation. The system worked by rendering both the score and spectrogram into an SVG, allowing direct interaction with the SVG file. The GUI also included an integrated XML text editor for direct manipulation of the SVG score code.

This experiment revealed significant complexity in creating interactive scores—both from UI/UX and technical perspectives. However, it provided invaluable insights into how Verovio and MEI function, as well as SVG structure, knowledge that would prove essential for the developed end solution. The project was ultimately abandoned as it did not produce a reusable Python library and the interaction features were not functional enough to justify further development in this direction.

3.2.3 BeatPrecision, a PP fork

In March 2025, we explored building on the PP and SV frameworks to create a BA GUI tool. The rationale was straightforward: PP already provided an embedded score alignment feature and all SV functionality, so adding a few BA-focused improvements seemed like a viable path to developing a relevant annotation tool. Using Qt/C++, we integrated BeatThis and implemented

automatic score follow-along in a horizontal manner to synchronize with SV's spectrogram and audio-visualizations.

While functional, the project revealed significant challenges. The SV/PP codebase proved complex and difficult to manage. Long compile times and the sheer size of the codebase made each iteration increasingly challenging to implement, manage and debug. More importantly, this iteration taught us a critical lesson: we needed to distinguish between the interaction and visualization problems in BA. We realized we could not tackle both simultaneously and would need to choose a direction. Recognizing a clear gap in the field, we opted to focus on the visualization aspect, as we saw greater potential for the thesis to benefit the community in this area.

3.2.4 Score-Alignment.py

By the end of March 2025, we shifted our focus to creating a Python-based visualization tool with score-aligned spectrograms. Our motivation was to concentrate on score alignment in a Python based environment, while exploring the Modusa framework's capabilities. Using Python, Modusa, and matplotlib, we built an interactive spectrogram.

However, this project was abandoned relatively quickly. Implementing a truly interactive and fluid spectrogram (with zoom, pan, timeline, and playback cursor) proved significantly more challenging than anticipated. While we managed to develop a functional interactive spectrogram, its implementation was clunky and unsatisfactory, leading us to discontinue the project in favor of other approaches.

3.2.5 Score-Alignment.js

In April 2026, we attempted to recreate the last iteration, this time implementing it in JS/HTML. Building on the experience from MEI-Basic-Edit and the initial BeatMapJS project, we aimed to create a comprehensive audio-to-score alignment with spectrogram visualization. Leveraging JS/HTML, Verovio/MEI, and Wavesurfer.js, we developed a system that displayed MEI scores horizontally alongside spectrograms. Timestamps (intended to representing beat annotations) could be directly stored in the MEI file using the `<when>` function, and notes in the score would highlight when hovered over, selected, or associated with a timestamp.

While not fully functional, the application showed promise. However, we made a deliberate decision to shift direction once again. We recognized the need to develop a strictly visualization tool purely in python-based with an emphasis on a OO approach that could be reused and extended by the community. This pivotal decision marked the beginning of LayerIt development.

3.3 LayerIt!

It was only by mid April 2026 that LayerIt started being developed. LayerIt, is intended to be a Python OO tool for visualizing recorded performances synchronized with the corresponding music score in a time-aligned manner. This tool allows for custom made visualizations of audio recordings where visual elements are treated as independent layers that can be copied, moved and combined in order to develop custom visualizations.

3.3.1 LayerIt Overview

The project bases itself in librosa [8], matplotlib [14] and modusa [9] to generate audio and data visualizations. The score layer is rendered through ScoreWarp. The end result is an SVG file that overlays the audio and data layers and with the score. The codebase also integrates BeatThis the as-of-now SOTA BT algorithm. Through its implementation we demonstrate how a given BT model can be integrated to our system in order to produce beat data visualizations.

3.3.2 Choices Made and Why

LayerIt relies on matplotlib for audio and data representations where, the SVG format is the end visual format. The reason, stems from the fact that Verovio (and subsequently ScoreWarp) render the score in SVG, and because matplotlib does not provide seamless integration of SVG charts, it is more reliable to export matplotlib visuals into a SVG framework, then the other way around.

The SVG format, is a vector-based one, therefore, data that can be mathematically represented in cartesian plots (i.e., X/Y graphs) can be exported without losing resolution. This is the case for most of MIR algorithmic data visualizations i.e., continuous curves and time based annotations. However, some data types, cant be presented in this way noticeably, heatmap-based plots like spectrograms. For interaction proposes, these usually are rendered in real-time, allowing for seamless navigation (i.e., zooming and panning). This is not possible since SVG is an inherently static format, therefore plots like these need to be rasterized before being integrated into the SVG. To circumvent this, we employed a way to render plots as PNGs as well. This trade-off means that cartesian plots and the score can be zoomed-in infinitely while always preserving the highest level of fidelity and accuracy. On the other hand, rasterized layers are bound to lose quality with zooming while, at the same time, also requiring heavier processing and memory usage.

That being said, because SVG is a text markup format, it can be easily accessed and modified, allowing it to be and intermediate format that can bridge different codebases independently of their base language (i.e., Python, JS, C++ etc...). This also serves as a crucial value argument for this choice, it being: the development and integration of LayerIt into future MIR applications.

Our main goal is that, this codebase may be easily integrated into other relevant projects to generate visualizations. Hence why we developed it as a Python Based 00 tool that is based on commonly used MIR libraries such as librosa and matplotlib. This way, community integration and development is eased.

3.4 Chapter Summary

In this chapter, we presented LayerIt, a Python-based OO tool designed to address a critical gap in the MIR community: the lack of a unified framework for generating time-aligned visualizations of audio, music scores, and algorithmic data. We began by identifying the core problem—that researchers face a disconnect between modern GUI tools like SV and programming-based workflows, forcing many to develop *ad hoc* solutions for visualization tasks.

We traced the development journey of LayerIt through five distinct iterations, each providing valuable insights that shaped the final solution and thesis theme. From web-based beat annotation (BeatMapJS and Score-Alignment.js) and interactive score editing (MEI-Basic-Edit), to attempts at extending existing frameworks (BeatPrecision) and a Python-based approach (Score-Alignment.py), we learned critical lessons about the distinction between interaction design and visualization challenges. This iterative process led us to focus exclusively on the visualization aspect, resulting in a Python-based, modular architecture that prioritizes extensibility and community integration.

We justified key design decisions, particularly the adoption of SVG as the primary output format, which balances the need for scalable vector graphics for score and analytical data with practical rasterization of spectrograms. Finally, we established LayerIt's value proposition: a reusable, object-oriented tool built on established MIR libraries (librosa, matplotlib, ScoreWarp) that can be seamlessly integrated into existing research workflows and future frameworks.

Chapter 4

Implementation of LayerIt!

In this chapter, we will explore more in depth LayerIt, the core project of this thesis. We will explore the developed architecture, and we'll be providing a broad clarification on how the system works. By the end of this chapter, the reader should better understand the value of the developed approach, as well as how it can be further developed and integrated with other systems.

4.1 Generating Visuals

LayerIt was developed with the goal of generating visualizations for BA and BT tasks, therefore, the current version focuses on tools for these purposes. Hence the reason we focus so much in providing tools for audio-to-score and data alignment tools, since we believe that this is a core gap for annotation and evaluation tasks for BT and BA tasks and subsequently for MIR as a whole.

LayerIt provides tools for visualizing BT/BA relevant data, therefore in it's current state the codebase relies on the following files to generate complete visuals:

File	Description
`.wav`	File with the audio recording of the performance
`.mei`	XML Score of the musical piece
`.maps.json`	Contains data that links onsets with score elements of the .mei
`.svg`	The score in visual format generated by ScoreWarp (requires .mei and .maps *a priori*)
`.npz`	File with algorithmic BT data

Table 4.1: Files required by LayerIt to generate complete visuals.

4.1.1 Getting the Warped Score

The warped score can be generated using the ScoreWarp online demo. In it, one needs to upload the score in .mei format and the MAPS file in order to download the score in svg format.

The .mei file that contains the score in xml format. The easiest way to get any .mei score is to get the score in .musicxml format and then convert it to .mei, this can be done in multiple

ways we will describe two: The first being through the verovio online editor. The second, by using musescore's software that allows to then convert to .mei. In either way, it is important to get the original score in .musicxml format to be able to then convert it to .mei using the tools we described.

4.1.2 Getting the MAPS file

The MAPS file, describes the onsets of the performance and links them to each corresponding score elements of the .mei score through their @xml.id . Getting the maps file might be a bit more cumbersome, but we can describe two different ways of obtaining it:

The first is the one described in [4], using the MIDI-to-MIDI matching algorithm by [11]. However, this method requires , along side with the .mei score, the performance in midi format. Using examples from the asap dataset [12] will facilitate this process, but if a midi performance is not possible to obtain then the maps file needs to be generated independently.

The MAPS file is at it's core a .json file, and for the purposes of the current LayerIt version, it needs to provide the score's and performance onset data in the following manner:

Listing 4.1: Minimal structure of the MAPS.json file to work with LayerIt and ScoreWarp

```
{
    "obs_mean_onset": 0.853333333,
    "xml_id": ["x1iw1eq1"],
    "obs_num": 1
}
```

Where:

- "*obs_mean_onset*" is the time in seconds of a given onset.
- "*xml_id*" is the @xml.id of the corresponding onset note in the .mei score. In the case that the onset has more than one note, like a chord hit, this attribute can have more than one @xml.id.
- "*obs_num*" acts as a simple counter for each provided onset.

LayerIt relies on MAPS in two complementary ways: directly for onset visualization, and indirectly through ScoreWarp, which requires MAPS to generate the warped score aligned with the timeline SVG element which constitutes the alignment reference for all overlaid data.

ScoreWarp performs best when MAPS contains dense onset annotations across the full performance. However, it can still align scores when annotations are sparse via interpolation between available onsets and their @xml.id anchor points. In practice, this allows different precision levels (i.e., downbeats, phrase boundaries, or measure-level anchors), trading annotation effort for alignment precision. Even minimal anchors (first and last onset) may be usable for short performances or segments, but intermediate alignment becomes less reliable due to interpolation error.

For the examples that we provide in the making of our codebase we modified PP to obtain this data. PP's "Piano Aligner" plugin [5] matches performance onsets to score elements defined by the .mei file. In PP these are displayed in a regular SV time instance layer. We made a simple modification where, this layer, instead of displaying each onset with it's corresponding score position, "*measure + position_in_measure*", it would display the @xml.id of each onset. With this we could then export the time instances as .txt files with "*obs_mean_onset+xml_id*", then convert it into the MAPS format using "*TXT_to_Maps()*" function in @src/functions/warp_score.

4.1.3 Providing BT data

Since contemporary BT algorithms are ML based, they primarily function by generating a beat activation function, which is a continuous curve representing the probability of a beat occurring at any given frame. Visualizing this underlying probability curve, rather than just the discrete beat assessments, allows us to see the algorithm's internal confidence level, giving crucial insights on the inner workings of the model.

In @src/functions/Beat_Layers, the function "*Run_BeatThis()*" generates an .npz file that contains the required data for this to be later visualized. This particular function creates the .npz file generated through BeatThis. However, the .npz file format can serve as an intermediate for BT visual layers, as long as it is structured like it is described in "*Run_BeatThis()*". The .npz should contain five key components from the BT model:

Field	Purpose
`beat_times`	Time coordinates (seconds) aligned with probability frames
`beat_probs`	Continuous beat activation function values
`downbeat_probs`	Continuous downbeat activation function values
`detected_beats`	Discrete beat positions derived from peak detection
`detected_downbeats`	Discrete downbeat positions from peak detection

The probabilistic fields capture the algorithm's internal confidence across time, while the detected positions represent the final beat assessments. This separation enables visualization of both the continuous confidence landscape and discrete beat estimates.

4.2 Layer Alignment Method

As of now, LayerIt provides tools for generating visualizations with BT and BA tasks in mind, however, this is not an exclusive application of the codebase. Layers (be it data or audio representations) are simply either SVG or PNG elements that span the same or a proportional width to a given timeline, therefore, users can upload any SVG or PNG plots, given that they follow the same timeline length.

When the warped score is generated, ScoreWarp provides a functionality to add a visual timeline of the performance. This timeline will most likely span a wider length than the actual recorded performance length, possibly even getting out of bounds to the visual SVG canvas. Yet, since this

timeline provides the time reference for positioning the score elements correctly, it is also used in LayerIt to align different visual layers to that same recording.

The way LayerIt aligns the layers is by:

1. Looking at the SVG file containing the Warped Score (rendered through Score Warp) and retrieving the full length of the generated timeline in pixels.
2. Getting the actual audio recording duration (in time) within that same timeline.
3. Using a simple rule of three, it calculates the recording length (in pixels) by comparing the recording's duration in time to the full timeline length in pixels.

In the codebase this is done through a single method:

- `Visualizer().get_Layers_WidthHeight()`

Although LayerIt does not yet integrate methods to convert visualizations outside of what it already provides, the basic methodology within the layer creation is this. Image files only need to be placed in the same x position of the start of the related timeline and the method to get the length is already in the codebase. Any given image format, should just respect that same recording length and then scale it to the appropriate timeline length of the score generated through ScoreWarp.

4.3 Layers on Layers, within Layers

At its core, LayerIt provides a way to combine aligned visualizations of algorithmic data, audio and symbolic music representations (i.e., scores) that share the same timeline. The true value of our approach comes from the Python OO layer-based methodology that we developed in our approach. This allows not only the visualizations to be deeply customizable, but also, for implementing with other tools and workflows.

The visualizations generated by LayerIt follow a "layers on layers, within layers" philosophy: visual elements are self-contained components that can be composed into upper-level elements, which in turn can be composed into yet higher levels. Figure 4.1 demonstrates this methodology.

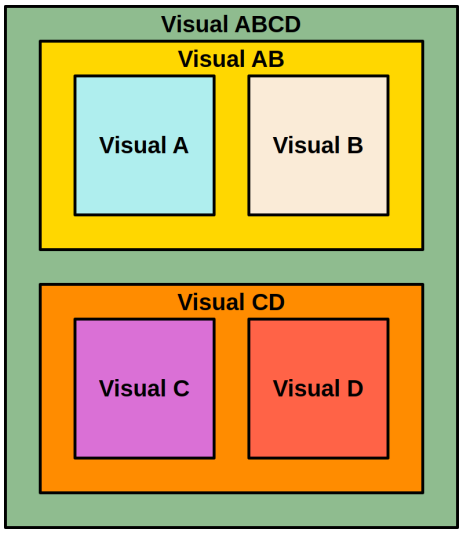

Figure 4.1: Composition of LayerIt visualization system. Each colored box represents a distinct visual element that compose higher-level visualizations.

This "layers within layers" design philosophy is applied at three different levels within the project's codebase: the **visual level**, where visualizations are composed of aligned data, audio and musical elements; the **SVG level**, in which LayerIt organizes the XML structure of the SVG; to the **code level**, with it's OO architecture of the codebase. The strength of this layer-based approach lies in the flexible foundation it provides, leaving the codebase open to future development and integration with other tools, workflows, and projects.

4.3.1 Visual Level

The visual level of this "Layers within Layers" philosophy refers to the actual visual output of LayerIt. Where layers are visual elements that are aligned within themselves and between themselves.

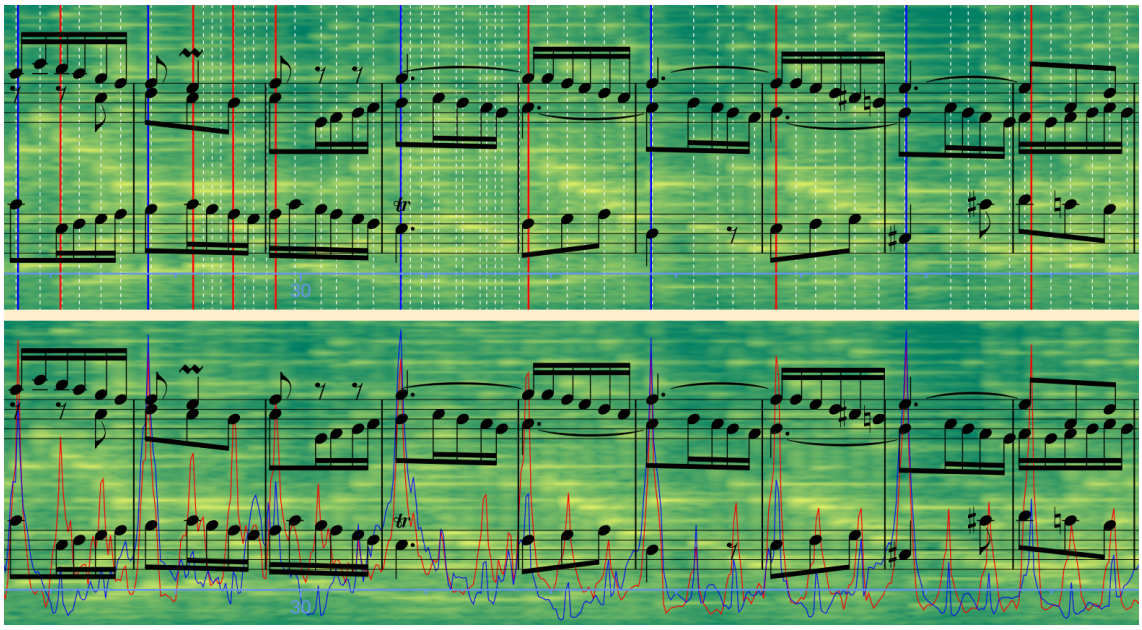

Figure 4.2: LayerIt generated visualization example for comparing the end output of BeatThis (on top) and the probability layer (at the bottom). In blue downbeats, in red beats and the doted white lines show onsets.

In Figure 4.2 we can see two distinct visualizations of the same recording. Both have distinct data layers that compose the top and bottom visualizations of the same performance, the top one provides onset and BeatThis unaltered output. The bottom one, shows BeatThis assessment of a frame-by-frame curve with the beat positions probability. Visually we can identify multiple layers that compose the individual visualizations: the spectrogram layers, score layers and respective data layers, we can also identify the two upper-level aligned layers that compose the whole example image.

Side note: Figure 4.2 is a particularly good example that demonstrates the capability of this kind of visualization for annotation and evaluation for BT purposes. On the top example we can see that BeatThis misses some beats and downbeats. However, on the bottom one, we can see that the model actually shows probability peaks that widely correspond to the actual beat positions. This type of visualization shows a concrete example for evaluating (in this case) the BeatThis model that SV simply cant provide in this way. For annotation purposes we could use this kind of visualization to effectively see missing or wrongly annotated beats/downbeats.

4.3.2 SVG Level

This refers to the way LayerIt organizes the XML structure of the SVG. Where each layer (combined or individual layers) follow a structured and organized approach. The following listing shows a simplified representation from the XML from the same SVG shown in Figure 4.2:

Listing 4.2: Example structure of an SVG file

```
<svg>
|   <Top Visualization>
|   |   <Layers>
|   |   |   <image spectrogram.png>
|   |   |   <BeatThis Output>
|   |   |   <Onsets>
|   |   <Score>
|   |   <timeAxis>
|   <Bottom Visualization>
|   |   <Layers>
|   |   |   <image spectrogram.png>
|   |   |   <Beat Probability>
|   |   |   <Downbeat Probability>
|   |   <Score>
|   |   <timeAxis>
</svg>
```

This kind of XML organization makes it easier to manipulate the SVG directly through it's embedded code. This allows for another level of visualization customization (through directly changing the XML text) and also should make it more simple to be accessed via code-based solutions, easing the integration with other projects while also maintaining the SVG readability. Yet once again, we can see our layer based organization method in action, where elements are put inside each other at the XML level as well.

4.3.3 Code Level

At this level we refer to the actual OO architecture of the codebase. LayerIt's OO architecture is built on a principle of modular composability: each visualization component is a self-contained, reusable module that adheres to a consistent interface. This enables the same "layers within layers" philosophy that characterizes the visual and SVG levels to emerge naturally at the code level. The following snippets illustrate the code-level contract (base abstraction), the Visualizer composition API, and a short end-to-end workflow.

Listing 4.3 shows the 'Layer' base-class interface (illustrative). For clarity the required imports are included.

Listing 4.3: Layer interface (concise)

```
class Layer(ABC):
  def __init__(self, name: str = "Layer"):
    self._data = None
    self.name = name

  @abstractmethod
  def load_data(self, **kwargs) -> bool:
```

```
    ...

  @abstractmethod
  def draw(self, ax, shared_data) -> tuple:
    ...
```

In LayerIt, a Visualizer instance composes multiple lower-level layers, which may represent data plots or audio representations, and provides them with a shared data dictionary via .load_all_layers(). The next listing (4.5) shows how a Visualizer instance should be instantiated.

Listing 4.4: Visualizer Instance Composition

```
# Creating a visualizer and composing layers
Layer1 = Visualizer()

# Add multiple layers - they're composable
Layer1.add_layer(BeatProbabilityLayer(color=(1, 0, 0)))
Layer1.add_layer(DownbeatProbabilityLayer(color=(0, 0, 1)))
# ...

# Load all data and render the combined layers
Layer1.load_all_layers(audio_path="performance.wav",
              beat_file="beats.npz")
fig, ax = Layer1.draw()
```

After this, each Visualizer instance can be rendered to an SVG or PNG. At this stage, Visualizer outputs are still Matplotlib figures and can be displayed directly with .show(). However, if they are to be combined with other Visualizer instances and a score, they need to be exported to SVG. This intermediate SVG layer allows the aligned score to be added on top of each visualization, producing a higher-level composite that can then be stacked with other rendered SVGs to create the final aligned view. The following listing shows exactly that.

Listing 4.5: Simplified workings for SVG layers.

```
# Turn the Visualizer into SVG and aligned it to the warped score
SVG_Layer1 = Layer1.turn_to_SVG("Output/Path/And/Filename", svg_warped_score)
SVG_Layer2 = Layer2.turn_to_SVG("Output/Path/And/Filename", svg_warped_score)

# Add Score to a given SVG Layer
SVGLayer1_withScore = Layer1.combine_layers_with_score(filename, svg_warped_score)

# Create the final SVG with each SVGLayer Stacked and place their relative vertical
    positions.
EndVisualization = Visualizer()
svg_layers = [(SVG_Layer1_withScore, 50),(SVG_Layer1, 200), (SVG_Layer2, 350) ...]
```

```
11 EndVisualization.create_final_SVG(width, height, (...), svg_layers)
```

This shows that higher-level SVG outputs can be positioned or stacked vertically, while lower-level layers (like Matplotlib layers) can be arranged in the same flexible way. That compositional behavior reinforces the "layers within layers" philosophy that underpins LayerIt.

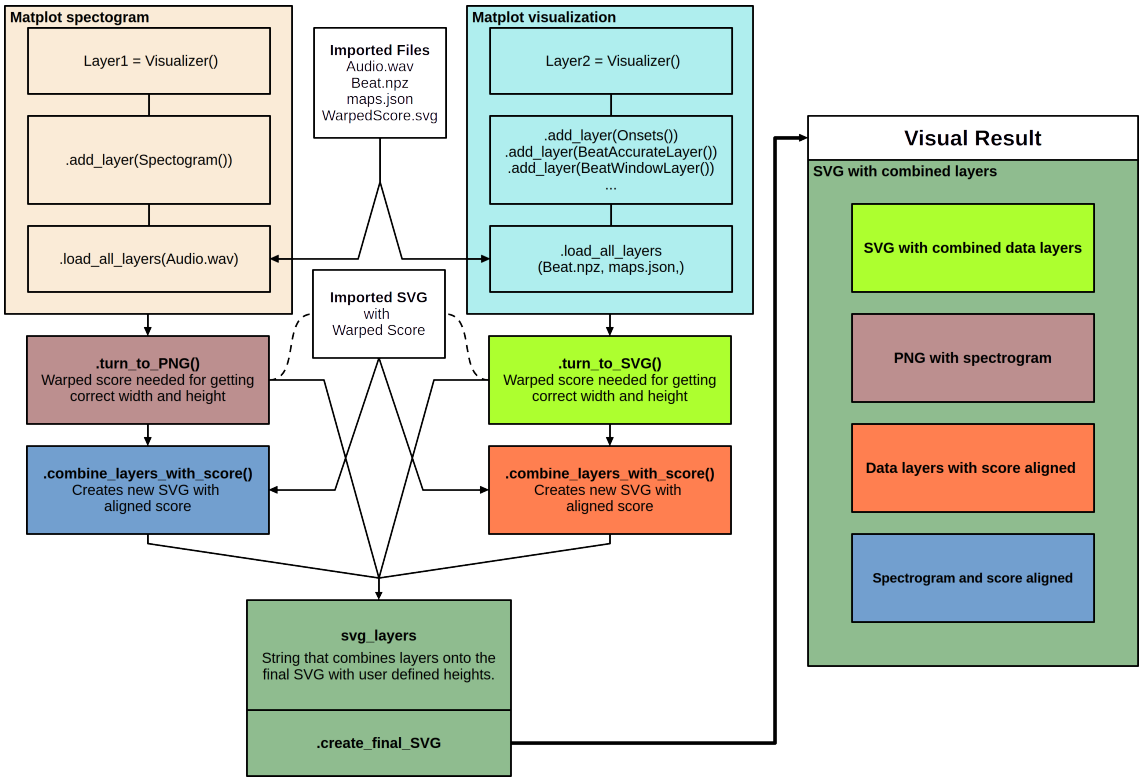

Figure 4.3: Complete LayerIt workflow: from Visualizer composition to final stacked SVG visualization. The diagram shows how multiple visualization layers are created independently, exported to SVG format, combined with the warped score, and finally stacked vertically with defined y-offsets to produce the final custom composite visualization.

The architecture of LayerIt defines different levels of interaction, from Matplotlib-based generation of visualizations to this SVG/PNG methodology, which adds another layer of flexibility and integration. This final layer allows users to incorporate externally produced graphics, enabling the system to combine code-generated visualizations with images created outside the codebase.

This hybrid design is intentional: it makes LayerIt useful for multiple workflows, from Python-based prototyping to GUI-centric and third-party visualization pipelines. By supporting both custom layer implementations and external image inputs, LayerIt provides a practical bridge between research code and real-world visualization needs.

These code-level choices make LayerIt more than a set of rendering tools: they make it a platform for experimentation and integration. That platform-level flexibility is what allows new visualization types to be added without changes to the core framework, supporting the extensibility requirements outlined in chapter 3 and positioning LayerIt as a foundation for future MIR

visualization work.

4.4 Chapter Conclusion

This chapter presented LayerIt, a system for generating time-aligned visualizations of audio, music scores, and algorithmic data. We demonstrated how LayerIt addresses the core challenge of integrating multiple data modalities (i.e., audio, symbolic notation, and beat tracking outputs) into cohesive visualizations suitable for beat annotation and beat tracking evaluation tasks.

A central contribution of this chapter is the articulation of LayerIt's design philosophy: the "layers within layers" approach, which manifests consistently across three complementary levels. At the **visual level**, independent visualization components are composed into higher-order visualizations. At the **SVG level**, this compositionality is preserved in the XML structure, enabling direct manipulation and external integration. Critically, at the **code level**, the same principle is realized through an object-oriented architecture based on modular abstraction and composability, where visualization components adhere to a shared interface and can be freely combined without modification to the core orchestration logic.

This unified architectural approach yields two key benefits: *Practical functionality*, demonstrated through successful integration of spectrograms, beat tracking data with aligned scores; and *Extensibility*, which ensures that new visualization types can be added independently, and the system can be adapted to diverse MIR workflows and integrated into future frameworks. The choice to output SVG, which is a text-based, language-agnostic format, further reinforces this goal of seamless integration with the broader MIR ecosystem.

LayerIt thus fulfills the thesis objectives outlined in chapter 3: it bridges the gap between GUI-based tools and code-based workflows, provides a reusable Python foundation for the community, and establishes an architecture explicitly designed for extensibility and integration. By grounding these capabilities in a coherent, reproducible technical design, LayerIt provides both immediate value for BA and BT visualization tasks and a foundation upon which future MIR visualization tools can build.

Chapter 5

Conclusions and Future Work

This final chapter reflects on the thesis journey and the LayerIt framework developed throughout this work. We begin by reviewing the key achievements: how the identified gaps in MIR visualization have been addressed and what conceptual contributions this thesis makes to the field. We then outline realistic, community-oriented directions for future development that preserve LayerIt's architectural vision while expanding its scope and accessibility. Finally, we reflect on the broader significance of this work within the evolving MIR research landscape.

5.1 Achievements

This thesis set out to address a fundamental gap in the MIR community: the absence of a unified, extensible framework for generating time-aligned visualizations that integrate audio, scores, and algorithmic data. Through the development of LayerIt, we have successfully demonstrated a solution that bridges this gap and establishes a foundation for future work in MIR visualization.

The key achievements of this thesis are:

1. **Bridged the GUI-to-Code Workflow Gap** — We identified and formalized a critical disconnect between GUI-based tools like SV and contemporary code-based research workflows. LayerIt provides a programmatic alternative that allows researchers to generate sophisticated multi-modal visualizations without relying on manual manipulation or the limitations of existing GUI frameworks. This directly addresses the problem outlined in chapter 3.

2. **Unified Multi-Modal Alignment** — LayerIt successfully integrates audio spectrograms, symbolic music representation (via ScoreWarp), and algorithmic beat tracking data into a single time-coherent visualization. By leveraging the warped score's timeline as a temporal anchor, we demonstrated a robust method for aligning diverse data modalities across different time domains.

3. **Demonstrated Modular OO Architecture** — We developed and validated an architecture based on modular composability and abstract interfaces. This design enables visualization

components to be self-contained, reusable, and freely composable without modification to the core orchestration logic. We showed that this same philosophy can be consistently applied across three levels: visual, SVG, and code, creating a coherent and intuitive system.

4. **SVG-First Output Strategy** — By choosing SVG as the primary output format, we positioned LayerIt for seamless integration with other tools and workflows. SVG's text-based, language-agnostic nature enables direct manipulation, external editing, and integration into web-based or GUI frameworks developed in other languages.

5. **Integrated Contemporary Beat Tracking** — We successfully demonstrated the integration of BeatThis, a state-of-the-art ML-based beat tracking algorithm, into LayerIt's visualization pipeline. Critically, we showed how visualizing both discrete beat estimates and continuous probabilistic outputs provides unprecedented insight into model behavior and confidence—revealing opportunities for improved evaluation and annotation workflows.

6. **Established Community-Ready Foundation** — LayerIt is built on widely-adopted MIR libraries (librosa, matplotlib, ScoreWarp) using standard Python conventions, lowering the barrier for community adoption and extension. The modular design explicitly supports new visualization types and integration into existing research workflows.

7. **Validated Iterative Development Approach** — Through five distinct project iterations (documented in chapter 3), we identified core lessons about the distinction between interaction design and visualization challenges, ultimately focusing on the visualization problem where we could provide maximum value to the community.

These achievements collectively demonstrate that the original thesis objective—developing a reusable, extensible framework for MIR visualization—has been met and validated through a working implementation.

5.2 Future Work

While LayerIt represents a significant step forward, the framework opens multiple avenues for future development. The following directions represent natural evolutionary extensions that preserve LayerIt's architectural vision while expanding scope, accessibility, and applicability:

1. **Interactive Visualization Interfaces** — LayerIt currently focuses on batch visualization and static SVG output. Future work should explore building interactive frontends leveraging LayerIt's SVG output—web-based viewers or desktop applications with graphical parameter controls. Such interfaces would complete the vision initiated in earlier thesis iterations (BeatMapJS, MEI-Basic-Edit) and provide researchers with fluid, exploratory visualization experiences.

2. **Expanded Visualization Coverage** — While LayerIt was developed with beat tracking and beat annotation tasks in mind, the underlying framework is task-agnostic. Future work should extend coverage to other MIR problems that benefit from aligned multi-modal visualization: onset detection, structural segmentation, instrument recognition, polyphonic transcription, and performance expressive analysis. Each would contribute new specialized layer types while reinforcing the generality of the core architecture.

3. **Integration with Existing Ecosystems** — Tighter integration with established tools would accelerate adoption. Direct plugins or wrapper libraries for SV, PP, and Jupyter notebooks would reduce friction for researchers already embedded in these workflows. Similarly, standardized APIs for integration with librosa and other MIR libraries would enable LayerIt to become a natural component in larger analysis pipelines.

4. **Collaborative Annotation Frameworks** — Extend LayerIt's visualization foundation into annotation and correction interfaces. Build on the layer-based architecture to create collaborative workflows where multiple annotators can provide beat annotations, corrections, or confidence assessments. Integrate human-in-the-loop machine learning approaches (as in [16]) to provide intelligent annotation suggestions and improve annotation efficiency.

5. **Comparative Algorithm Visualization** — Currently LayerIt demonstrates BeatThis integration. Extend support to enable comparative visualization of multiple beat tracking algorithms on the same performance. This would allow researchers to directly compare algorithmic behavior, identify failure modes, and benchmark different approaches. Thus, addressing an important gap in contemporary BT evaluation practices.

6. **Performance Optimization and Scalability** — Optimize LayerIt for large-scale datasets and real-time processing pipelines. Implement streaming visualization for long recordings, support GPU-accelerated spectrogram computation, and develop memory-efficient layer management. These enhancements would enable LayerIt to support production-scale research workflows and large-scale evaluation studies.

7. **Standardization and Community Standards** — Formalize and propose community standards for score-performance alignment metadata (building on the MAPS format), facilitating ecosystem interoperability. Collaborate with initiatives like TROMPA, the Music Information Retrieval Evaluation eXchange (MIREX) to establish conventions that enable tool-agnostic data sharing across the research community.

Each of these directions represents incremental progress toward a more comprehensive MIR visualization ecosystem, while maintaining fidelity to LayerIt's core design principles: modularity, composability, extensibility, and community integration.

5.3 Final Remarks

This thesis began with the observation of a critical gap in the MIR research community: the absence of a unified framework for generating time-aligned, multi-modal visualizations suited to contemporary research workflows. Through an iterative development process and careful architectural design, we have delivered LayerIt, a Python-based, object-oriented tool that addresses this gap while explicitly supporting future extension and integration.

LayerIt's value lies not merely in its immediate applicability to beat tracking and annotation tasks, but in its demonstration that visualization frameworks can be architected for extensibility, composability, and community adoption. By grounding the system in established conventions (OO design patterns, common MIR libraries, standardized formats), we have created a foundation upon which the community can build up on.

As MIR research continues to evolve, driven by advances in machine learning, increasing dataset complexity, and growing interdisciplinary collaboration. The need for sophisticated, flexible visualization tools will only intensify. LayerIt is positioned to serve as a foundation for this future work, enabling researchers to focus on their core innovations rather than reimplementing visualization infrastructure from scratch.

References

[1] *An Introduction to MEI*. URL: https://music-encoding.org/about/.

[2] J J Carabias-Orti et al. "AN AUDIO TO SCORE ALIGNMENT FRAMEWORK USING SPECTRAL FACTORIZATION AND DYNAMIC TIME WARPING". In: ().

[3] Zhiyao Duan and Bryan Pardo. "Soundprism: An Online System for Score-Informed Source Separation of Music Audio". In: *IEEE Journal of Selected Topics in Signal Processing* 5.6 (Oct. 2011), pp. 1205–1215. ISSN: 1932-4553, 1941-0484. DOI: 10.1109/JSTSP.2011.2159701. URL: http://ieeexplore.ieee.org/document/5887382/ (visited on 06/07/2026).

[4] Werner Goebl. "Let's Do the ScoreWarp Again! Shifting Notes to Performance Timelines". In: (2025).

[5] Yucong Jiang and Werner Goebl. "PIANO PRECISION: ADVANCING MUSIC PERFORMANCE ANALYSIS BY INTEGRATING USER-CORRECTABLE AUDIO-TO-SCORE ALIGNMENT". In: (2025).

[6] Jongmin Jung et al. "Unified Cross-modal Translation of Score Images, Symbolic Music, and Performance Audio". In: *IEEE Transactions on Audio, Speech and Language Processing* 34 (2026), pp. 1876–1891. ISSN: 2998-4173. DOI: 10.1109/TASLPRO.2025.3648794. arXiv: 2505.12863 [cs.SD]. URL: http://arxiv.org/abs/2505.12863 (visited on 06/02/2026).

[7] Zulfidin Khodzhaev. "A Practical Guide to Spectrogram Analysis for Audio Signal Processing". In: ().

[8] Brian McFee et al. *Librosa/Librosa: 0.11.0*. Version 0.11.0. Zenodo, Mar. 11, 2025. DOI: 10.5281/ZENODO.15006942. URL: https://zenodo.org/doi/10.5281/zenodo.15006942 (visited on 06/22/2026).

[9] meluron. *Modusa*. Version 6.3.7. Feb. 2026. URL: https://github.com/meluron/modusa.

[10] Meinard Müller. *Fundamentals of Music Processing: Audio, Analysis, Algorithms, Applications*. Cham: Springer International Publishing, 2015. ISBN: 978-3-319-21944-8 978-3-319-21945-5. DOI: 10.1007/978-3-319-21945-5. URL: https://link.springer.com/10.1007/978-3-319-21945-5 (visited on 06/02/2026).

[11] Eita Nakamura, Kazuyoshi Yoshii, and Haruhiro Katayose. "PERFORMANCE ERROR DETECTION AND POST-PROCESSING FOR FAST AND ACCURATE SYMBOLIC MUSIC ALIGNMENT". In: ().

[12] Silvan David Peter et al. “Automatic Note-Level Score-to-Performance Alignments in the ASAP Dataset”. In: *Transactions of the International Society for Music Information Retrieval* 6.1 (June 26, 2023), pp. 27–42. ISSN: 2514-3298. DOI: 10.5334/tismir.149. URL: http://transactions.ismir.net/articles/10.5334/tismir.149/ (visited on 06/07/2026).

[13] Répertoire International des Sources Musicales. *Verovio Reference Book*. 2021. DOI: 10.5448/7EM6-MY23. URL: https://book.verovio.org/ (visited on 06/06/2026). Pre-published.

[14] The Matplotlib Development Team. *Matplotlib: Visualization with Python*. Version v3.10.9. Zenodo, Apr. 24, 2026. DOI: 10.5281/ZENODO.19716234. URL: https://zenodo.org/doi/10.5281/zenodo.19716234 (visited on 06/22/2026).

[15] Rafael Valle. “Visual Display and Retrieval of Music Information”. In: ().

[16] K. Yamamoto. “Human-in-the-Loop Adaptation for Interactive Musical Beat Tracking”. In: *Proceedings of the 22nd International Society for Music Information Retrieval Conference (ISMIR)*. 2021.